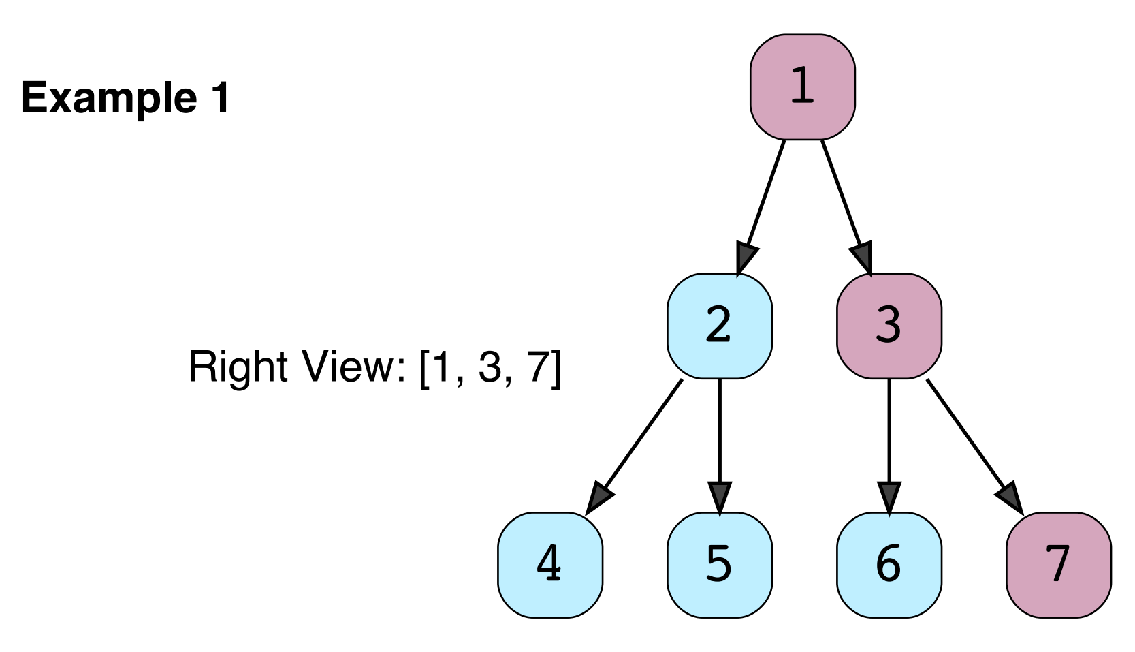
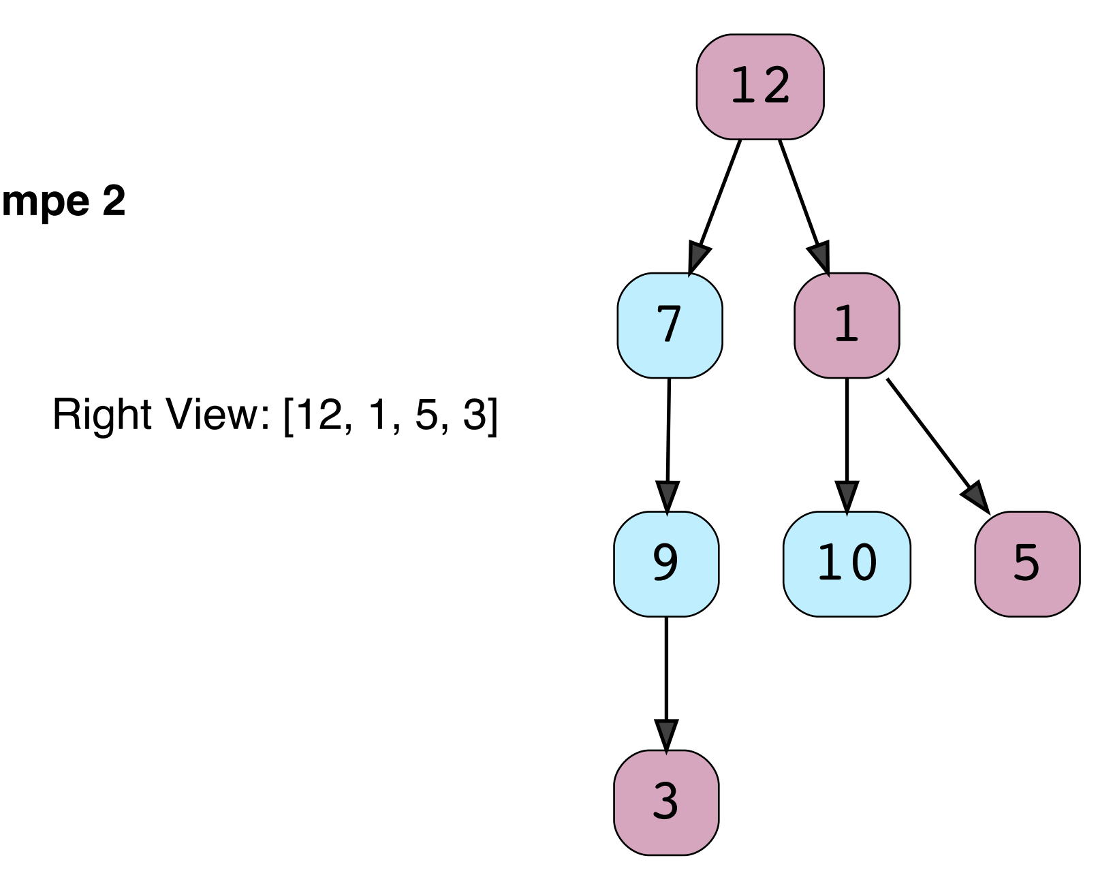
Solution: Problem Challenge 2: Right View of a Binary Tree

Problem Statement

Given a binary tree, return an array containing nodes in its right view. The right view of a binary tree is the set of **nodes visible when the tree is seen from the right side.**





Solution

This problem follows the **Binary Tree Level Order Traversal** pattern. We can follow the same **BFS** approach. The only additional thing we will be doing is to append the last node of each level to the result array.

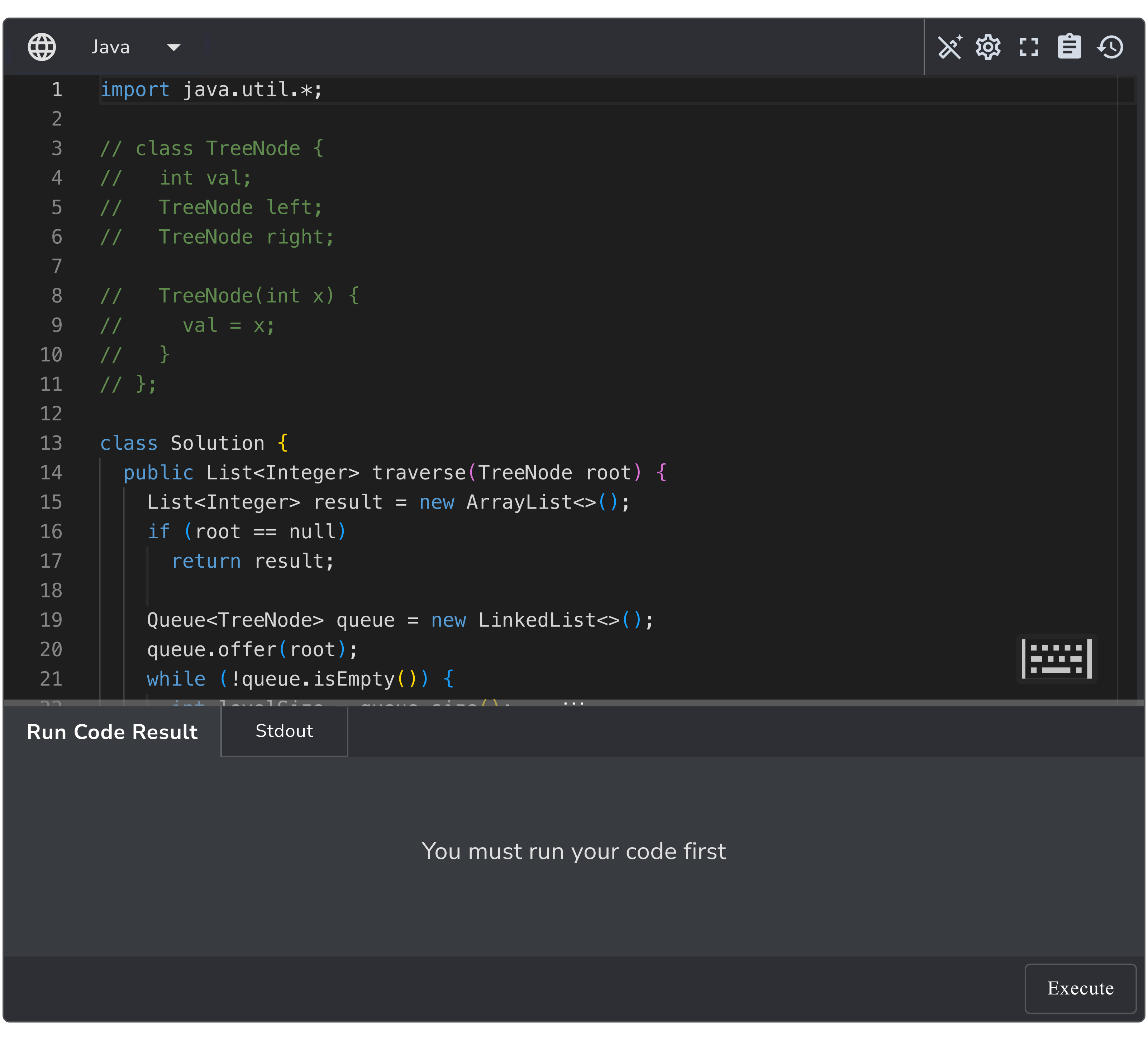
Here is the visual representation of the algorithm:

Level Size: 1
Queue:
12
1
Insert
Level Size: 2
Queue:
1
Dequeue
Right View Tree
Since 12 is the only node at the level 1 so it is added in the Right View tree
2
Insert
7 is not the last node at level 2 so it will not be included in the right view tree
Level Size: 3
Since 1 is the last node at level 2 so it is added in the right view tree
Since 5 is the last node at the level 3 so it is added in the Right View tree
Since 3 is the only node at the level 4 so it is added in the Right View tree
7
9
10
5
3

Right View of BT

Code

Here is the code for this algorithm:



```java
import java.util.*;

// class TreeNode {
//   int val;
//   TreeNode left;
//   TreeNode right;

//   TreeNode(int x) {
//     val = x;
//   }
// };

class Solution {
  public List<Integer> traverse(TreeNode root) {
    List<Integer> result = new ArrayList<>();
    if (root == null)
      return result;

    Queue<TreeNode> queue = new LinkedList<>();
    queue.offer(root);
    while (!queue.isEmpty()) {
```

You must run your code first

Time Complexity

The time complexity of the above algorithm is O(N), where ‘N’ is the total number of nodes in the tree. This is due to the fact that we traverse each node once.

Space Complexity

The space complexity of the above algorithm will be O(N) needed for the return list. We will also need O(N) space for the queue. Since we can have a maximum of N/2 nodes at any level (this could happen only at the lowest level), therefore we will need O(N) space to store them in the queue.

Similar Questions

Problem 1: Given a binary tree, return an array containing nodes in its left view. The left view of a binary tree is the set of nodes visible when the tree is seen from the left side.

Solution: We will be following a similar approach, but instead of appending the last element of each level, we will be appending the first element of each level to the output array.

← Previous
Problem Challenge 2: Right View Of A Binary Tree (Easy)

Next →
Introduction To Tree Depth First Search Pattern

☐ Mark as Completed